

Genesys Cloud Conversation Timeline Analysis

Purpose

This document defines how to combine a targeted set of Genesys Cloud API endpoints to produce a high-fidelity timeline analysis for either:

1. A single conversation.
2. A collection of conversations, such as the last 7 days of interactions for one agent.

The goal is not merely to list API responses. The goal is to normalize multiple Genesys data surfaces into one chronological, engineer-friendly timeline that explains:

- When the conversation started and ended.
- Which participants were involved.
- Which agent, queue, flow, media type, and routing path were involved.
- Which segments occurred and in what order.
- Where alert, talk, hold, ACW, IVR, transfer, consult, disconnect, error, recording, speech analytics, suggestion, and custom attribute events fit into the conversation.
- Which metadata explains the “why” behind the observed timeline.

This document is written for a Genesys.Core-style architecture where API extraction should be centralized in the core/runtime layer, and UI applications should consume normalized run artifacts rather than implement raw REST, pagination, polling, or enrichment logic directly.

Endpoint Set

The proposed timeline analyzer uses the following endpoint groups.

Endpoint	Primary Role	Use in Timeline Analysis
<code>POST /api/v2/analytics/conversations/details/query</code>	Synchronous conversation detail search	Fast preview, small result sets, initial validation, short intervals.
<code>POST /api/v2/analytics/conversations/details/jobs</code>	Asynchronous conversation detail search	Preferred bulk extraction path for multi-day or high-volume analysis.
<code>GET /api/v2/analytics/conversations/details/jobs/{jobId}</code>	Job status	Required after job creation; poll until completion/failure.

Endpoint	Primary Role	Use in Timeline Analysis
<code>GET /api/v2/analytics/conversations/details/jobs/{jobId}/results</code>	Job results	Required to retrieve the async conversation detail result set.
<code>GET /api/v2/conversations/{conversationId}</code>	Conversation runtime/detail object	Enrichment layer for participants, sessions, attributes, live-ish conversation shape, and fields not convenient in analytics result.
<code>POST /api/v2/conversations/customattributes/search</code>	Cross-conversation custom attribute search	Bulk lookup/indexing of custom attributes for many conversations.
<code>GET /api/v2/conversations/{conversationId}/customattributes</code>	Single-conversation custom attributes	Per-conversation fallback/enrichment for custom attributes.
<code>POST /api/v2/conversations/participants/attributes/search</code>	Historical participant attribute search	Bulk lookup of participant data attributes, especially IVR-collected data and Architect flow data.
<code>GET /api/v2/conversations/{conversationId}/suggestions</code>	Agent assist / suggestion listing	Adds suggestion events to the timeline.
<code>GET /api/v2/conversations/{conversationId}/suggestions/{suggestionId}</code>	Suggestion detail	Enriches individual suggestions with full content/detail.
<code>GET /api/v2/conversations/{conversationId}/recordingmetadata</code>	Recording metadata	Adds recording availability, recording IDs, media metadata, and related timing context.
<code>GET /api/v2/speechandtextanalytics/conversations/{conversationId}</code>	Speech and text analytics summary	Adds transcript/sentiment/topic/communication analytics context where available.

Recommended Architecture

Core Principle

The UI should not call these endpoints directly.

A timeline-capable application should call a single Genesys.Core dataset or command such as:

```
Invoke-Dataset -DatasetKey analytics-conversation-timeline-analysis -Parameters
@{
    AgentId = "<agent-user-id>"
    Interval = "2026-04-30T00:00:00.000Z/2026-05-07T00:00:00.000Z"
    IncludeConversationObject = $true
    IncludeCustomAttributes = $true
    IncludeParticipantAttributes = $true
    IncludeSuggestions = $true
    IncludeRecordingMetadata = $true
    IncludeSpeechTextAnalytics = $true
}
```

The core/runtime should own:

- Authentication.
- Region/base URL selection.
- Rate limiting.
- Retry behavior.
- Pagination.
- Async job creation, status polling, and results retrieval.
- Per-conversation enrichment fan-out.
- Artifact persistence.
- Error capture and partial-result handling.

The UI should consume artifacts only.

Output Artifacts

A mature implementation should emit a run folder similar to this:

```
runs/
  conversation-timeline-analysis/
    2026-05-07T21-45-00Z_agent-<agentId>/
      manifest.json
      summary.json
      errors.jsonl
      conversations.jsonl
      timeline-events.jsonl
      timeline-index.json
      data/
        analytics-details.jsonl
        conversation-objects.jsonl
        custom-attributes.jsonl
        participant-attributes.jsonl
```

```
suggestions.jsonl
recording-metadata.jsonl
speech-text-analytics.jsonl
reports/
  timeline-analysis.xlsx
  timeline-analysis.html
  conversation-timeline-summary.md
```

manifest.json

Run metadata and reproducibility record.

```
{
  "datasetKey": "analytics-conversation-timeline-analysis",
  "runId": "2026-05-07T21-45-00Z_agent-1234",
  "region": "usw2.pure.cloud",
  "interval": "2026-04-30T00:00:00.000Z/2026-05-07T00:00:00.000Z",
  "agentId": "1234",
  "createdUtc": "2026-05-07T21:45:00.000Z",
  "requestedEnrichments": {
    "conversationObject": true,
    "customAttributes": true,
    "participantAttributes": true,
    "suggestions": true,
    "recordingMetadata": true,
    "speechTextAnalytics": true
  },
  "sourceEndpoints": [
    "POST /api/v2/analytics/conversations/details/jobs",
    "GET /api/v2/analytics/conversations/details/jobs/{jobId}",
    "GET /api/v2/analytics/conversations/details/jobs/{jobId}/results",
    "GET /api/v2/conversations/{conversationId}",
    "POST /api/v2/conversations/customattributes/search",
    "POST /api/v2/conversations/participants/attributes/search",
    "GET /api/v2/conversations/{conversationId}/suggestions",
    "GET /api/v2/conversations/{conversationId}/recordingmetadata",
    "GET /api/v2/speechandtextanalytics/conversations/{conversationId}"
  ]
}
```

summary.json

Aggregated metrics for dashboard and executive summary.

```
{
  "conversationCount": 412,
  "agentId": "1234",
  "interval": "2026-04-30T00:00:00.000Z/2026-05-07T00:00:00.000Z",
  "mediaTypes": {
    "voice": 388,
    "callback": 17,
    "message": 7
  },
  "disconnectTypes": {
    "client": 210,
    "endpoint": 120,
    "transfer": 38,
    "error": 4
  },
  "timelineEventCount": 9217,
  "conversationsWithRecording": 301,
  "conversationsWithSpeechAnalytics": 244,
  "conversationsWithSuggestions": 67,
  "conversationsWithCustomAttributes": 199,
  "partialEnrichmentCount": 18,
  "errorCount": 3
}
```

timeline-events.jsonl

The normalized event stream used by the UI.

```
{
  "conversationId": "abc",
  "eventTime": "2026-05-06T14:01:11.200Z",
  "eventType": "conversation.start",
  "source": 10,
  "label": "Conversation started",
  "details": {
    "originatingDirection": "inbound",
    "mediaType": "voice"
  }
}, {
  "conversationId": "abc",
  "eventTime": "2026-05-06T14:01:13.300Z",
  "eventType": "segment.ivr",
  "source": 20,
  "label": "IVR segment",
  "details": {
    "flowName": "Main IVR",
    "segmentType": "ivr"
  }
}, {
  "conversationId": "abc",
  "eventTime": "2026-05-06T14:02:01.500Z",
  "eventType": "segment.alert",
  "source": 30,
  "label": "Agent alerting",
  "details": {
    "userId": "agent-1234",
    "queueId": "queue-1",
    "durationMs": 9000
  }
}, {
  "conversationId": "abc",
  "eventTime": "2026-05-06T14:02:10.500Z",
  "eventType": "segment.talk",
  "source": 40,
  "label": "Agent connected / talk",
  "details": {
    "userId": "agent-1234",
    "durationMs": 240000
  }
}, {
  "conversationId": "abc",
  "eventTime": "2026-05-06T14:03:20.000Z",
  "eventType": "speech.sentiment",
  "source": 45,
  "label": "Negative sentiment detected",
  "details": {
    "text-analytics": {
      "participantId": "p1",
      "sessionId": "s1",
      "sequence": 1
    },
    "sentiment": "negative",
    "confidence": 0.82
  }
}
```

```
{"conversationId": "abc", "eventTime": "2026-05-06T14:06:11.000Z", "eventType": "recording.available", 80, "label": "Recording metadata available", "details": {"recordingId": "rec-123"}}
```

Query Strategy

Preview Mode

Use `POST /api/v2/analytics/conversations/details/query` when:

- The user is testing filters.
- The interval is small.
- The expected result set is limited.
- The UI needs fast feedback before starting a bulk run.

Example request for a quick preview of conversations involving one agent:

```
{
  "interval": "2026-05-06T00:00:00.000Z/2026-05-07T00:00:00.000Z",
  "order": "asc",
  "orderBy": "conversationStart",
  "segmentFilters": [
    {
      "type": "and",
      "predicates": [
        {
          "type": "dimension",
          "dimension": "userId",
          "operator": "matches",
          "value": "<agent-user-id>"
        }
      ]
    }
  ],
  "paging": {
    "pageSize": 100,
    "pageNumber": 1
  }
}
```

Important behavior: the synchronous detail query interval includes conversations that started on a day touched by the interval. That makes it useful for targeted search, but it can be awkward when investigating conversations with activity that spills across interval boundaries.

Bulk Mode

Use `POST /api/v2/analytics/conversations/details/jobs` when:

- Pulling multiple days.
- Pulling a full 7-day history for one agent.
- Pulling many queues, divisions, media types, or agents.
- You need more robust extraction with async job handling.

Example request for the last 7 days for one agent:

```
{
  "interval": "2026-04-30T00:00:00.000Z/2026-05-07T00:00:00.000Z",
  "order": "asc",
  "orderBy": "conversationStart",
  "segmentFilters": [
    {
      "type": "and",
      "predicates": [
        {
          "type": "dimension",
          "dimension": "userId",
          "operator": "matches",
          "value": "<agent-user-id>"
        }
      ]
    }
  ],
  "startOfDayIntervalMatching": true
}
```

Important behavior: the async job interval is better aligned to historical extraction because results include conversations that had activity during the interval. That makes it preferable for timeline reconstruction.

Required Async Job Workflow

The async job endpoint alone is not sufficient. The full workflow is:

1. `POST /api/v2/analytics/conversations/details/jobs`
-> returns `jobId`
2. `GET /api/v2/analytics/conversations/details/jobs/{jobId}`
-> poll status until fulfilled/completed or failed
3. `GET /api/v2/analytics/conversations/details/jobs/{jobId}/results`
-> retrieve result pages

4. Persist raw job results to analytics-details.jsonl

5. Begin enrichment phase by conversationId

If an app only creates the job and never retrieves status/results, it will correctly authenticate and possibly report that a run completed, but it will not have conversation rows to display.

Data Model

Conversation Record

Each conversation should be normalized into a parent record.

```
{
  "conversationId": "abc",
  "conversationStart": "2026-05-06T14:01:11.200Z",
  "conversationEnd": "2026-05-06T14:06:11.000Z",
  "originatingDirection": "inbound",
  "mediaTypes": ["voice"],
  "agentIds": ["agent-1234"],
  "queueIds": ["queue-1"],
  "divisionIds": ["division-1"],
  "disconnectTypes": ["client"],
  "hasRecording": true,
  "hasSpeechAnalytics": true,
  "hasSuggestions": false,
  "enrichmentStatus": {
    "conversationObject": "complete",
    "customAttributes": "complete",
    "participantAttributes": "complete",
    "suggestions": "none",
    "recordingMetadata": "complete",
    "speechTextAnalytics": "complete"
  }
}
```

Timeline Event

Every endpoint-specific artifact should be converted into normalized timeline events.

```
{
  "conversationId": "abc",
```



```

"eventTime": "2026-05-06T14:02:10.500Z",
"eventType": "segment.talk",
"source": "analytics.details",
"participantId": "p2",
"sessionId": "s2",
"communicationId": null,
"sequence": 40,
"label": "Agent talk segment",
"details": {
  "userId": "agent-1234",
  "queueId": "queue-1",
  "segmentType": "interact",
  "durationMs": 240000
},
"rawRef": {
  "artifact": "data/analytics-details.jsonl",
  "line": 187
}
}

```

Event Ordering Rules

Sort timeline events by:

1. `eventTime` ascending.
2. `eventPriority` ascending.
3. `participantPurpose` priority.
4. `source` priority.
5. Stable ingestion order.

Recommended event priorities:

Priority	Event Class
10	Conversation start/end
20	Participant/session creation
30	IVR/flow/routing events
40	Queue/alert/answer/talk/hold/ACW segments
50	Transfer/consult/conference/coaching/barge/monitor events
60	Participant/custom attributes
70	Suggestions/agent assist
80	Speech analytics/sentiment/topic events

Priority	Event Class
90	Recording metadata
100	Disconnect/end/wrap-up
900	Warnings, enrichment gaps, inferred events

Endpoint-by-Endpoint Usage

1. `POST /api/v2/analytics/conversations/details/query`

Purpose

Synchronous conversation-detail search. This is the fastest way to validate filters and build a preview dataset.

Inputs

Important input fields:

- `interval` : ISO-8601 interval.
- `conversationFilters` : conversation-level filters.
- `segmentFilters` : segment-level filters.
- `evaluationFilters` : evaluation filters.
- `surveyFilters` : survey filters.
- `resolutionFilters` : resolution filters.
- `order` : `asc`, `desc`, or `unordered`.
- `orderBy` : `conversationStart`, `conversationEnd`, `segmentStart`, or `segmentEnd`.
- `paging` : `pageSize` and `pageNumber`.

Usage in Timeline Analysis

Use this endpoint to:

- Build a “Preview matching conversations” button.
- Validate `agentId`, `queueId`, `mediaType`, `direction`, or `conversationId` filters.
- Retrieve small result sets.
- Compare expected records before launching an async job.

Output Handling

Persist the raw response into:

```
data/analytics-details-preview.jsonl
```

Normalize each returned conversation into:

- One parent conversation record.
- Multiple timeline events derived from participants, sessions, and segments.

2. POST /api/v2/analytics/conversations/details/jobs

Purpose

Asynchronous historical conversation-detail extraction.

Inputs

Similar to the synchronous query endpoint, but optimized for larger historical workloads.

Important input fields:

- `interval`.
- `conversationFilters`.
- `segmentFilters`.
- `evaluationFilters`.
- `surveyFilters`.
- `resolutionFilters`.
- `order`.
- `orderBy`.
- `limit`.
- `startOfDayIntervalMatching`.

Usage in Timeline Analysis

Use this as the primary extraction endpoint for:

- 7-day agent timelines.
- Queue-level historical timeline analysis.
- Multi-agent troubleshooting.
- Large enough datasets where pagination and request timeouts become concerns.

Output Handling

The create-job response should be stored in the run manifest:

```
{
  "analyticsDetailsJobId": "<job-id>",
  "jobCreatedUtc": "2026-05-07T21:45:10.000Z"
}
```

The job result rows should be persisted as:

```
data/analytics-details.jsonl
```

3. GET /api/v2/analytics/conversations/details/jobs/{jobId}

Purpose

Checks async job status.

Usage in Timeline Analysis

This endpoint is required between job creation and result retrieval.

The runtime should poll using bounded retry/backoff until the job reaches a terminal state:

- Complete / fulfilled / succeeded: retrieve results.
- Failed: persist failure details and stop extraction.
- Expired / canceled: mark run failed.
- Still running: continue polling within configured timeout.

Output Handling

Append each poll result to:

```
data/job-status.jsonl
```

Also update `manifest.json` with final job status.

4. GET /api/v2/analytics/conversations/details/jobs/{jobId}/results

Purpose

Retrieves the async job result set.

Usage in Timeline Analysis

This is the endpoint that actually produces the conversation detail rows after the async job completes.

A correct implementation must:

- Retrieve all result pages.
- Persist each conversation result.
- Extract `conversationId` values for enrichment.
- Continue gracefully if some enrichment calls fail later.

Output Handling

Persist each result as a line in:

```
data/analytics-details.jsonl
```

Then create:

```
timeline-index.json
```

with a list of conversation IDs and enrichment status.

5. GET /api/v2/conversations/{conversationId}

Purpose

Retrieves the canonical conversation object for one conversation.

Usage in Timeline Analysis

Use this as a per-conversation enrichment endpoint.

It is useful for:

- Confirming participant shape.
- Retrieving participant-level attributes attached to the conversation object.
- Inspecting sessions and communications.
- Filling gaps where analytics details are optimized for reporting rather than object inspection.
- Debugging why a UI has a conversation row but no detail pane.

Output Handling

Persist each response as:

```
data/conversation-objects.jsonl
```

Normalize additional events such as:

- `participant.created`.
- `participant.attribute`.
- `session.created`.
- `communication.created`.
- `conversation.state`.

6. `POST /api/v2/conversations/customattributes/search`

Purpose

Searches custom attributes across conversations.

Usage in Timeline Analysis

Use this when analyzing a set of conversations and you need custom attributes in bulk.

Examples:

- External case ID.
- CRM record ID.
- Customer type.
- Business unit.
- Call reason.
- Integration correlation ID.

Input Notes

The uploaded endpoint shape includes:

```
{
  "expand": [
    "",
  ],
  "pageSize": 0,
  "pageNumber": 0,
  "sort": [
    {
      "sortOrder": "",
      "sortBy": ""
    }
  ],
  "sortBy": "",
  "sortOrder": ""
}
```

The `expand` options should be verified directly in the current Developer Center/API Explorer before implementation. Until confirmed, the safest contract is:

- Treat `expand` as optional.
- Do not send empty-string expand values.
- Log the resolved schema/options during catalog generation.
- Prefer a minimal valid request first, then add expansion only when proven.

Output Handling

Persist results as:

```
data/custom-attributes.jsonl
```

Normalize each attribute into a timeline event:

```
{
  "eventType": "attribute.custom",
  "source": "customattributes.search",
  "eventTime": "<conversationStart-or-attribute-timestamp-if-available>",
  "label": "Custom attribute: CaseId",
  "details": {
    "name": "CaseId",
    "value": "CASE-12345"
  }
}
```

If the attribute response has no specific event timestamp, attach the event to `conversationStart` and mark it as metadata rather than a true chronological event.

7. `GET /api/v2/conversations/{conversationId}/customattributes`

Purpose

Retrieves custom attributes for a single conversation.

Usage in Timeline Analysis

Use this as:

- A fallback when bulk custom attribute search does not return a specific conversation.
- A direct per-conversation enrichment path when analyzing only one conversation.
- A detail-pane endpoint when the user clicks a conversation.

Output Handling

Write to the same artifact as bulk custom attributes, but include source metadata:

```
{
  "conversationId": "abc",
  "sourceEndpoint": "GET /api/v2/conversations/{conversationId}/
customattributes",
  "attributes": {}
}
```

8. `POST /api/v2/conversations/participants/attributes/search`

Purpose

Searches participant attributes across historical conversations.

Participant data attributes are especially important because they often explain what happened in Architect flows and IVR steps.

Examples:

- Account number entered.
- Menu path selected.
- Data action result.
- Customer validation status.
- Routing decision metadata.
- External system lookup result.

Usage in Timeline Analysis

Use this endpoint after the analytics conversation list has been retrieved.

The best workflow is:

1. Extract conversation IDs from analytics details.
2. Search participant attributes for those conversations or the same interval.
3. Join attributes by conversationId and participantId.
4. Place attributes near the relevant participant/session/flow segment in the timeline.

Output Handling

Persist as:

```
data/participant-attributes.jsonl
```

Normalize into events:

```
{
  "conversationId": "abc",
  "participantId": "p1",
  "eventType": "attribute.participant",
  "source": "participants.attributes.search",
  "eventTime": "2026-05-06T14:01:13.300Z",
  "label": "Participant attribute: Authenticated",
  "details": {
    "name": "Authenticated",
    "value": "true"
  }
}
```

Timestamp rule:

- If the attribute has an explicit timestamp, use it.
- If it belongs to a participant/session but has no timestamp, anchor it to the earliest known segment for that participant.
- If neither is available, anchor it to `conversationStart` and mark it as metadata.

9. `GET /api/v2/conversations/{conversationId}/suggestions`

Purpose

Lists suggestions associated with a conversation.

Usage in Timeline Analysis

Use this for conversations where Agent Assist, knowledge suggestions, bots, or suggestion surfaces may have contributed to the agent experience.

The timeline can show:

- Suggestion offered.
- Suggestion viewed or accepted, if available.
- Suggestion source.
- Suggested knowledge/article/intent.

Output Handling

Persist as:

```
data/suggestions.jsonl
```

Normalize into:

```
{
  "eventType": "suggestion.offered",
  "source": "conversation.suggestions",
  "eventTime": "<suggestion timestamp>",
  "label": "Suggestion offered",
  "details": {
    "suggestionId": "sug-123",
    "title": "Password reset article"
```

```
}  
}
```

If the list endpoint returns only summary rows, retrieve detail using the next endpoint.

10. GET /api/v2/conversations/{conversationId}/ suggestions/{suggestionId}

Purpose

Retrieves full detail for a single suggestion.

Usage in Timeline Analysis

Use this only when:

- The summary row does not contain enough detail.
- The user opens a conversation detail pane.
- The report requires suggestion content, confidence, source, or article metadata.

Avoid calling this for every suggestion by default in very large runs unless the use case requires full fidelity.

Output Handling

Append enriched suggestion detail to:

```
data/suggestions.jsonl
```

The timeline event should retain both:

- Summary fields from the list endpoint.
 - Full detail fields from the detail endpoint.
-

11. GET /api/v2/conversations/{conversationId}/ recordingmetadata

Purpose

Retrieves recording metadata for a conversation.

Usage in Timeline Analysis

Use this endpoint to determine:

- Whether a recording exists.
- Recording IDs.
- Media metadata.
- Recording-related timestamps where available.
- Whether downstream recording/transcript workflows are possible.

Important Annotation Note

Do not rely on recording metadata as the long-term source of recording annotations. Treat annotations as a separate concern and use the recording annotations endpoint if annotation detail is required.

Output Handling

Persist as:

```
data/recording-metadata.jsonl
```

Normalize into events:

```
{
  "eventType": "recording.available",
  "source": "recordingmetadata",
  "eventTime": "2026-05-06T14:06:11.000Z",
  "label": "Recording metadata available",
  "details": {
    "recordingId": "rec-123",
    "media": "audio"
  }
}
```

12. GET /api/v2/speechandtextanalytics/conversations/{conversationId}

Purpose

Retrieves speech and text analytics conversation data.

Usage in Timeline Analysis

Use this endpoint to enrich timelines with:

- Sentiment.
- Topics.
- Phrases.
- Communication analytics.
- Transcript availability or transcript references, depending on response shape and permissions.

Output Handling

Persist as:

```
data/speech-text-analytics.jsonl
```

Normalize into events such as:

```
{
  "eventType": "speech.topic",
  "source": "speech-text-analytics",
  "eventTime": "2026-05-06T14:03:22.000Z",
  "label": "Topic detected: Billing issue",
  "details": {
    "topic": "Billing issue",
    "confidence": 0.91
  }
}
```

and:

```
{
  "eventType": "speech.sentiment",
  "source": "speech-text-analytics",
  "eventTime": "2026-05-06T14:04:00.000Z",
  "label": "Negative sentiment detected",
  "details": {
    "sentiment": "negative",
    "score": -0.54
  }
}
```

Ideal Timeline Build Process

Phase 1: Select Scope

Inputs:

```
{
  "agentId": "<agent-user-id>",
  "interval": "2026-04-30T00:00:00.000Z/2026-05-07T00:00:00.000Z",
  "mediaTypes": ["voice", "callback", "message"],
  "includeEnrichment": true
}
```

Phase 2: Run Analytics Extraction

Decision rule:

```
If expected result count is small:
  Use POST /analytics/conversations/details/query
Else:
  Use POST /analytics/conversations/details/jobs
  Poll GET /jobs/{jobId}
  Retrieve GET /jobs/{jobId}/results
```

For a 7-day agent analysis, default to async jobs.

Phase 3: Persist Raw Analytics Details

Do not normalize directly in memory only. Persist raw source rows first.

```
data/analytics-details.jsonl
```

This gives the user and support engineer an audit trail.

Phase 4: Extract Conversation Index

Create a conversation index:

```
{
  "conversationId": "abc",
  "conversationStart": "2026-05-06T14:01:11.200Z",
```

```
"conversationEnd": "2026-05-06T14:06:11.000Z",  
"agentIds": ["agent-1234"],  
"queueIds": ["queue-1"],  
"mediaTypes": ["voice"],  
"needsEnrichment": true  
}
```

Phase 5: Enrich Conversations

For each conversation ID, retrieve optional enrichment data.

Recommended enrichment order:

1. GET /conversations/{conversationId}
2. GET /conversations/{conversationId}/customattributes
3. GET /conversations/{conversationId}/recordingmetadata
4. GET /speechandtextanalytics/conversations/{conversationId}
5. GET /conversations/{conversationId}/suggestions
6. GET /conversations/{conversationId}/suggestions/{suggestionId}, if needed

For participant attributes and custom attributes across many conversations, prefer bulk search endpoints when they can satisfy the need:

```
POST /conversations/participants/attributes/search  
POST /conversations/customattributes/search
```

Phase 6: Normalize Timeline Events

Create normalized events from each source.

Analytics details provide the backbone:

- Conversation start/end.
- Participant/session/segment path.
- Queue, flow, agent, IVR, routing, transfer, hold, talk, ACW, disconnect, wrap-up.

Conversation object adds object-level context:

- Participant shape.
- Session and communication details.
- Additional attributes.

Custom attributes add business context:

- CRM IDs.
- External correlation IDs.
- Business categorization.

Participant attributes add flow/IVR context:

- Architect variables.
- Collected digits.
- Data action outcomes.

Suggestions add agent-assist context:

- Recommendations offered during the interaction.

Recording metadata adds recording context:

- Recording IDs and availability.

Speech/text analytics adds content intelligence:

- Sentiment.
- Topics.
- Phrases.
- Transcript references where available.

Phase 7: Generate Analysis Layers

A useful analyzer should generate several views from the same timeline event stream.

Conversation Detail View

For one conversation:

```
14:01:11 Conversation started inbound voice
14:01:13 Customer entered IVR flow: Main IVR
14:01:34 Flow outcome: Authenticated = true
14:01:45 Routed to queue: Support Tier 1
14:02:01 Agent alerting: Ben Example
14:02:10 Agent answered
14:03:20 Negative sentiment detected
14:04:11 Hold started
14:04:45 Hold ended
14:05:30 Wrap-up selected: Billing Question
```


14:06:11 Conversation disconnected by client
14:06:11 Recording metadata available

Agent 7-Day Summary View

For a 7-day agent pull:

Metric	Description
Conversation count	Total conversations involving the agent.
Median handle time	Median duration from answer to ACW completion.
Longest conversations	Top outliers by total duration.
Longest holds	Top outliers by hold duration.
Transfer rate	Conversations with transfer-related segments.
Error/disconnect rate	Conversations with error-like disconnects or error codes.
Recording coverage	Conversations with recording metadata.
Speech analytics coverage	Conversations with speech/text analytics.
Suggestion usage	Conversations with suggestions present.
Attribute coverage	Conversations with custom/participant attributes.

Engineering Investigation View

For troubleshooting:

Signal	Why It Matters
Segment sequence	Shows exact call path and state transitions.
Queue and flow IDs	Helps isolate routing/Architect defects.
Disconnect type	Helps identify endpoint/client/system disconnect patterns.
SIP response code	Useful for telephony investigations when present.
Edge/provider/protocol IDs	Useful for trunk/SBC/provider correlation when present.
Wrap-up code/note	Helps map technical timeline to business outcome.
Participant attributes	Explains IVR and data-action decisions.
Speech analytics	Adds customer/agent experience context.

Recommended UI Behavior

Main Grid

Display one row per conversation:

Column	Purpose
Start	Conversation start time.
Duration	Total conversation duration.
Agent	Agent participant.
Queue	Queue path.
Direction	Inbound/outbound.
Media	Voice/callback/message/email/etc.
Disconnect	Final disconnect type.
Recording	Yes/no/unknown.
Speech Analytics	Yes/no/unknown.
Suggestions	Count.
Attributes	Count.
Flags	Errors, long hold, transfer, low MOS, missing enrichment.

Right-Hand Details Pane

When a user selects a conversation, show:

1. Header summary.
2. Timeline.
3. Participants and sessions.
4. Segment table.
5. Attributes.
6. Recording metadata.
7. Speech analytics.
8. Suggestions.
9. Raw source tabs.

Timeline Visualization

Recommended lanes:

```
Conversation
Customer
IVR / Flow
Queue / Routing
Agent
Recording
Speech Analytics
Suggestions
Metadata / Attributes
Warnings
```

This prevents the timeline from becoming a flat, unreadable event list.

Error Handling and Partial Results

The analyzer must support partial enrichment.

A conversation should still display if:

- Analytics details were retrieved.
- Recording metadata failed.
- Speech analytics was unavailable.
- Suggestions returned 404/no data.
- Custom attributes were empty.

Each enrichment call should write either:

```
{
  "conversationId": "abc",
  "sourceEndpoint": "GET /api/v2/conversations/{conversationId}/
recordingmetadata",
  "status": "success"
}
```

or:

```
{
  "conversationId": "abc",
  "sourceEndpoint": "GET /api/v2/conversations/{conversationId}/
recordingmetadata",
  "status": "failed",
  "httpStatus": 403,
}
```

```
"message": "Forbidden or missing permission"
}
```

Write failures to:

```
errors.jsonl
```

Then surface them in the UI as enrichment warnings, not total run failures.

Important Implementation Notes

transactionId vs jobId

Some Genesys workflows use a `transactionId` pattern, especially audit query workflows. Conversation detail jobs use a job workflow. Do not assume the audit query `transactionId` workflow and analytics detail job workflow are interchangeable.

For conversation timeline analysis, the async analytics flow should be modeled around:

```
POST details/jobs -> jobId
GET details/jobs/{jobId} -> status
GET details/jobs/{jobId}/results -> result pages
```

For audit analysis, model around:

```
POST audits/query -> transactionId
GET audits/query/{transactionId} -> status
GET audits/query/{transactionId}/results -> results
```

Both patterns are conceptually similar, but they should be represented distinctly in the catalog/runtime.

Preview vs Full Run

A good UX should include both:

- **Preview:** synchronous details query, limited rows, fast validation.
- **Full Run:** async details job, complete extraction, enrichment, artifacts.

Avoid Raw REST in App Wrappers

A WPF/MAUI/React UI should not contain Genesys pagination, job polling, or endpoint-specific request logic.

The wrapper should ask Genesys.Core for:

```
analytics-conversation-timeline-analysis
```

and then read artifacts.

Do Not Block Display on Enrichment Completion

The details grid should populate from analytics detail results first.

Then enrichment can update rows progressively:

```
Analytics rows loaded -> display immediately  
Conversation object enrichment -> update details pane  
Attributes -> update metadata tab  
Recording metadata -> update recording tab  
Speech analytics -> update insight tab  
Suggestions -> update suggestions tab
```

Acceptance Criteria

A correct implementation should meet these criteria:

1. Given an agent ID and 7-day interval, the runtime creates an async analytics detail job.
 2. The runtime polls job status until completion or terminal failure.
 3. The runtime retrieves all job result pages.
 4. The UI displays conversation rows from analytics details before enrichment completes.
 5. Each conversation has a normalized timeline generated from participants, sessions, and segments.
 6. Optional enrichment calls add attributes, suggestions, recording metadata, and speech/text analytics.
 7. Missing enrichment data produces warnings, not empty grids.
 8. Raw responses are persisted for audit/debugging.
 9. `timeline-events.jsonl` is the canonical UI event stream.
 10. The app contains no ad hoc REST extraction logic outside Genesys.Core.
 11. The right-hand details pane can render a selected conversation using only artifacts.
 12. The report/export contains summary, conversation list, timeline events, and enrichment coverage.
-

Recommended Dataset Contract

```
{
  "datasetKey": "analytics-conversation-timeline-analysis",
  "description": "Builds normalized timeline analysis for one or more Genesys Cloud conversations using analytics details plus optional enrichment endpoints.",
  "mode": "job-with-enrichment",
  "inputs": {
    "interval": { "type": "iso8601-interval", "required": true },
    "agentId": { "type": "uuid", "required": false },
    "conversationId": { "type": "uuid", "required": false },
    "queueId": { "type": "uuid", "required": false },
    "mediaTypes": { "type": "array", "required": false },
    "includeConversationObject": { "type": "boolean", "default": true },
    "includeCustomAttributes": { "type": "boolean", "default": true },
    "includeParticipantAttributes": { "type": "boolean", "default": true },
    "includeSuggestions": { "type": "boolean", "default": true },
    "includeRecordingMetadata": { "type": "boolean", "default": true },
    "includeSpeechTextAnalytics": { "type": "boolean", "default": true }
  },
  "outputs": {
    "manifest": "manifest.json",
    "summary": "summary.json",
    "conversations": "conversations.jsonl",
    "timelineEvents": "timeline-events.jsonl",
    "errors": "errors.jsonl",
    "rawDataDirectory": "data/"
  }
}
```

Recommended Next Coding-Agent Prompt

ROLE

You are a senior PowerShell/.NET engineer working on Genesys.Core and its thin UI wrappers.

OBJECTIVE

Implement a Genesys.Core dataset named analytics-conversation-timeline-analysis that builds normalized timeline artifacts for one or more Genesys Cloud conversations using analytics conversation details plus optional enrichment endpoints.

NON-NEGOTIABLE ARCHITECTURE

- Do not place raw Genesys REST extraction logic in the UI.
- All Genesys API calls, pagination, job polling, retries, and enrichment fan-out must live in Genesys.Core.
- UI wrappers consume run artifacts only.
- The canonical UI stream is timeline-events.jsonl.
- Persist raw source responses under data/ before normalization.

ENDPOINT WORKFLOW

Support these endpoint groups:

1. Preview mode:
 - POST /api/v2/analytics/conversations/details/query
2. Full run mode:
 - POST /api/v2/analytics/conversations/details/jobs
 - GET /api/v2/analytics/conversations/details/jobs/{jobId}
 - GET /api/v2/analytics/conversations/details/jobs/{jobId}/results
3. Per-conversation enrichment:
 - GET /api/v2/conversations/{conversationId}
 - GET /api/v2/conversations/{conversationId}/customattributes
 - GET /api/v2/conversations/{conversationId}/suggestions
 - GET /api/v2/conversations/{conversationId}/suggestions/{suggestionId}
 - GET /api/v2/conversations/{conversationId}/recordingmetadata
 - GET /api/v2/speechandtextanalytics/conversations/{conversationId}
4. Bulk enrichment where practical:
 - POST /api/v2/conversations/customattributes/search
 - POST /api/v2/conversations/participants/attributes/search

PRIMARY USE CASE

Given an agentId and a 7-day interval, retrieve every conversation involving that agent, then generate a normalized timeline analysis for each conversation.

OUTPUT ARTIFACTS

Create a run folder containing:

- manifest.json
- summary.json
- errors.jsonl
- conversations.jsonl
- timeline-events.jsonl
- timeline-index.json
- data/analytics-details.jsonl
- data/job-status.jsonl
- data/conversation-objects.jsonl
- data/custom-attributes.jsonl
- data/participant-attributes.jsonl
- data/suggestions.jsonl
- data/recording-metadata.jsonl
- data/speech-text-analytics.jsonl

TIMELINE REQUIREMENTS

Normalize all events into a stable schema:

- conversationId
- eventTime
- eventType
- source
- participantId
- sessionId
- communicationId
- sequence
- label
- details
- rawRef

Generate events for:

- conversation.start
- conversation.end
- participant/session creation where available
- IVR/flow/routing segments
- alert/talk/hold/ACW segments
- transfers/consults/conferences/coaching/monitor/barge where available
- disconnects and wrap-up data
- participant attributes
- custom attributes
- suggestions
- recording metadata
- speech analytics topics/sentiment/phrases where available
- enrichment warnings

ERROR HANDLING

- Analytics extraction failure fails the run.
- Enrichment failures do not fail the run.
- Missing optional data should produce enrichment warnings.
- Every failed enrichment call must be written to errors.jsonl with conversationId, endpoint, status code, and message.

ACCEPTANCE TESTS

1. A 7-day agent run creates an analytics detail job.
2. The job is polled until terminal status.
3. All result pages are retrieved.
4. conversations.jsonl is populated.
5. timeline-events.jsonl is populated and sorted by conversationId/eventTime/sequence.
6. A selected conversation can be rendered from artifacts without calling Genesys APIs from the UI.
7. If recording metadata or speech analytics returns no data/403/404, the conversation still displays.
8. The run summary reports coverage counts for recordings, speech analytics,

suggestions, custom attributes, participant attributes, and partial enrichment.

9. No UI file contains direct calls to /api/v2/analytics or /api/v2/conversations.
10. Existing Genesys.Core catalog/schema validation passes.